

Training Whisper for multilingual video transcription ▾

We have 4000+ videos to transcribe, the speaker is going to be same across the video and We anticipate Urdu / English and sometimes Arabic terms in the video. I would like to train the Whisper medium model, I have few questions, will training the model help in achieving higher accuracy when we transcribe after training? how much data do we need to train this model ?

Synthesized fine-tuning efficacy and data requirements for multilingual Whisper model >

Fine-tuning Whisper Medium for Your Use Case

Great questions! Here's a clear breakdown:

Will Fine-tuning Improve Accuracy?

Yes, significantly — especially for your specific scenario. Here's why:

Reasons it will help:

- **Code-switching** (Urdu ↔ English ↔ Arabic mid-sentence) is notoriously hard for generic models. Fine-tuning teaches the model *your* speaker's switching patterns.
- **Single speaker consistency** — accents, pronunciation quirks, and speaking style of one person become highly learnable.
- **Domain-specific vocabulary** — if the speaker uses technical, religious, or niche terminology, the base model will frequently mishear these. Fine-tuning corrects this.
- **Urdu specifically** — Whisper medium's Urdu performance is decent but not great out of the box, especially with heavy Roman Urdu or Nastaliq-adjacent phonetics.

Realistic accuracy gain: Expect **15-35% reduction in Word Error Rate (WER)** compared to the base model on your specific content.

How Much Data Do You Need?

Goal	Hours of Labeled Audio	Expected Outcome
Minimal fine-tune	1-3 hrs	Noticeable improvement on speaker accent & style
Good fine-tune	5-10 hrs	Strong improvement on code-switching & vocabulary
Solid fine-tune	10-20 hrs	Near-optimal for your domain
Diminishing returns beyond	20+ hrs	Marginal gains

For your case, 5-10 hours of high-quality transcribed audio is the sweet spot.

Data Quality Rules (Critical)

The data you use for fine-tuning must be:

- ✓ **Accurately transcribed** — errors in your training labels directly hurt the model
- ✓ **Phonetically diverse** — cover different topics/energy levels the speaker uses
- ✓ **Clean audio** — low background noise preferred; if your videos are noisy, include noisy samples too so the model learns your real conditions
- ✓ **Proper script** — decide upfront: are you transcribing Urdu in **Nastaliq** (اردو) or **Roman Urdu**? Be consistent.

Recommended Approach

1. **Start with** `whisper-medium` as your base (good balance of speed vs. accuracy for multilingual content)
2. **Manually transcribe 5-10 hours** from your existing 4000+ videos — pick diverse samples
3. **Fine-tune using Hugging Face** `transformers` + `seq2seq` **trainer** (well-documented pipeline)
4. **Evaluate WER** on a held-out test set before using it on the full 4000 videos
5. Once satisfied, **batch-transcribe** all 4000+ videos with your fine-tuned model

One Important Consideration

Since you have **4000+ videos** with the **same speaker**, you're actually in an ideal fine-tuning scenario. Even 3–5 hours of labeled data from *that specific speaker* will likely outperform what a much larger generic dataset would achieve. The single-speaker consistency is a major advantage.

Would you like guidance on the actual fine-tuning pipeline setup (data format, training script, hardware requirements)?

Yes, I would be using Claude Code to perform this POC and we are currently using Modal platform to train

Architected comprehensive Modal training pipeline with Claude Code integration >

Here's a comprehensive setup guide tailored for **Claude Code + Modal + Whisper medium** fine-tuning POC:

Project Structure

```
whisper-urdu-finetuning/
├── CLAUDE.md                ← Critical: Claude Code context file
├── requirements.txt
├── modal_app.py             ← Main Modal entry point
├── src/
│   ├── data_prep.py        ← Extract audio, chunk, validate
│   ├── dataset_builder.py   ← Build HuggingFace dataset
│   ├── train.py             ← Training logic (runs inside Modal)
│   ├── evaluate.py          ← WER computation
│   └── transcribe_batch.py  ← Bulk inference after fine-tuning
├── config/
│   └── training_config.yaml ← All hyperparameters in one place
└── scripts/
    ├── upload_data.py       ← Push prepared data to Modal Volume
    └── download_model.py    ← Pull fine-tuned model locally
```

1. **CLAUDE.md** — Claude Code Context File

This is the most important file for working with Claude Code. It gives Claude full project awareness.

markdown

Whisper Urdu Fine-tuning POC

Project Goal

Fine-tune OpenAI Whisper medium on a single Urdu/English/Arabic code-switching speaker for batch transcription of 4000+ videos.

Stack

- Training Platform: Modal (modal.com)
- Base Model: openai/whisper-medium
- Framework: HuggingFace Transformers + Seq2SeqTrainer
- Language: Python 3.11

Key Design Decisions

- Primary language token: Urdu (``<|ur|>``)
- Transcript script: Roman Urdu for English words, Nastaliq for Urdu, Arabic script kept for Arabic terms
- Audio format: 16kHz mono WAV, max 30s chunks
- Modal GPU: A10G (24GB VRAM) – fits Whisper medium with `batch_size=16`

Modal Resources

- App name: whisper-urdu-poc
- Volume name: whisper-training-vol (persists data + checkpoints)
- GPU: A10G for training, T4 for evaluation/inference

Commands

- Prepare data locally: ``python src/data_prep.py --input_dir ./raw_videos``
- Upload to Modal: ``python scripts/upload_data.py``
- Start training: ``modal run modal_app.py::train``
- Evaluate: ``modal run modal_app.py::evaluate``
- Batch transcribe: ``modal run modal_app.py::transcribe_batch``
- Download model: ``python scripts/download_model.py``

WER Targets

- Baseline (whisper-medium, no fine-tune): ~35-45% WER on this domain
- Target after fine-tuning: <20% WER

Data Format (HuggingFace datasets)

Each sample: { "audio": {"array": [...], "sampling_rate": 16000},
 "sentence": "transcription text",
 "language": "ur" }

2. `config/training_config.yaml`

yaml

```
model:
  name: "openai/whisper-medium"
  language: "ur"          # Primary language token
  task: "transcribe"

audio:
  sampling_rate: 16000
  max_duration_seconds: 30
  min_duration_seconds: 1

training:
  output_dir: "/data/checkpoints/whisper-medium-urdu"
  per_device_train_batch_size: 16
  per_device_eval_batch_size: 8
  gradient_accumulation_steps: 2   # Effective batch = 32
  learning_rate: 1.0e-5
  warmup_steps: 500
  max_steps: 5000
  gradient_checkpointing: true
  fp16: true
  evaluation_strategy: "steps"
  eval_steps: 500
  save_steps: 500
  save_total_limit: 3
  load_best_model_at_end: true
  metric_for_best_model: "wer"
  greater_is_better: false
  predict_with_generate: true
  generation_max_length: 225
  logging_steps: 25
  report_to: "tensorboard"
  logging_dir: "/data/logs"

data:
  train_split: 0.90
  eval_split: 0.10
  dataset_path: "/data/processed/dataset"
```

3. `src/data_prep.py` — Local Audio Preparation

```
python

"""
Run locally with Claude Code before uploading to Modal.
Extracts audio from videos, chunks them, validates quality.
"""

import os
import json
import subprocess
import argparse
from pathlib import Path
from dataclasses import dataclass, asdict


@dataclass
class Sample:
    audio_path: str
    transcript: str
    duration: float
    language: str = "ur"


def extract_audio(video_path: str, output_path: str) -> bool:
    """Extract 16kHz mono WAV from video using ffmpeg."""
    cmd = [
        "ffmpeg", "-i", video_path,
        "-ar", "16000",          # 16kHz sample rate (Whisper requirement)
        "-ac", "1",             # Mono
        "-c:a", "pcm_s16le",     # 16-bit PCM
        output_path,
        "-y", "-loglevel", "error"
    ]
    result = subprocess.run(cmd, capture_output=True)
    return result.returncode == 0


def get_duration(audio_path: str) -> float:
    """Get audio duration in seconds."""
    cmd = [
        "ffprobe", "-v", "error",
        "-show_entries", "format=duration",
        "-of", "default=noprint_wrappers=1:nokey=1",
        audio_path
    ]
    result = subprocess.run(cmd, capture_output=True, text=True)
    return float(result.stdout.strip())
```

```

def chunk_audio(audio_path: str, output_dir: str,
                max_duration: int = 28) -> list[str]:
    """
    Split audio into chunks of max_duration seconds.
    28s leaves buffer under Whisper's 30s limit.
    """
    Path(output_dir).mkdir(parents=True, exist_ok=True)
    duration = get_duration(audio_path)

    stem = Path(audio_path).stem
    chunks = []

    if duration <= max_duration:
        # Short file, no chunking needed
        out = f"{output_dir}/{stem}_000.wav"
        subprocess.run(["cp", audio_path, out])
        chunks.append(out)
    else:
        num_chunks = int(duration // max_duration) + 1
        for i in range(num_chunks):
            start = i * max_duration
            out = f"{output_dir}/{stem}_{i:03d}.wav"
            cmd = [
                "ffmpeg", "-i", audio_path,
                "-ss", str(start),
                "-t", str(max_duration),
                "-ar", "16000", "-ac", "1",
                out, "-y", "-loglevel", "error"
            ]
            subprocess.run(cmd)
            # Skip chunks shorter than 1 second
            if get_duration(out) >= 1.0:
                chunks.append(out)

    return chunks


def validate_transcript(transcript: str) -> bool:
    """Basic validation – customize for your use case."""
    if not transcript or len(transcript.strip()) < 2:
        return False
    if len(transcript) > 500: # Likely a bad transcript
        return False
    return True

```

```
def prepare_dataset(
    input_dir: str,
    transcript_dir: str, # Folder with .txt files matching audio names
    output_dir: str
):
    """
    Main preparation pipeline.
    Expects: input_dir/video.mp4 + transcript_dir/video.txt
    """
    audio_dir = Path(output_dir) / "audio"
    audio_dir.mkdir(parents=True, exist_ok=True)

    samples = []
    video_files = list(Path(input_dir).glob("*.mp4")) + \
                  list(Path(input_dir).glob("*.mkv")) + \
                  list(Path(input_dir).glob("*.webm"))

    print(f"Found {len(video_files)} video files")

    for video_path in video_files:
        transcript_path = Path(transcript_dir) / f"{video_path.stem}.txt"

        if not transcript_path.exists():
            print(f"  ▲ No transcript for {video_path.name}, skipping")
            continue

        transcript = transcript_path.read_text(encoding="utf-8").strip()

        if not validate_transcript(transcript):
            print(f"  ▲ Invalid transcript for {video_path.name}, skipping")
            continue

        # Extract audio
        wav_path = str(audio_dir / f"{video_path.stem}.wav")
        if not extract_audio(str(video_path), wav_path):
            print(f"  × Audio extraction failed: {video_path.name}")
            continue

        duration = get_duration(wav_path)

        # Chunk if longer than 28 seconds
        if duration > 28:
            chunks = chunk_audio(wav_path, str(audio_dir / "chunks"))
            # For chunked files, you need per-chunk transcripts
```



```

# This is a placeholder – handle per your transcript format
print(f"  i {video_path.name} is {duration:.1f}s, needs chunked trans
continue

samples.append(Sample(
    audio_path=wav_path,
    transcript=transcript,
    duration=duration
))
print(f"  ✓ {video_path.name} ({duration:.1f}s)")

# Save manifest
manifest_path = Path(output_dir) / "manifest.json"
with open(manifest_path, "w", encoding="utf-8") as f:
    json.dump([asdict(s) for s in samples], f, ensure_ascii=False, indent=2)

total_hours = sum(s.duration for s in samples) / 3600
print(f"\n✅ Prepared {len(samples)} samples ({total_hours:.2f} hours)")
print(f"  Manifest saved to: {manifest_path}")

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--input_dir", required=True)
    parser.add_argument("--transcript_dir", required=True)
    parser.add_argument("--output_dir", default="./data/processed")
    args = parser.parse_args()

    prepare_dataset(args.input_dir, args.transcript_dir, args.output_dir)

```

4. `modal_app.py` — The Core Modal Application

```

python

"""
Main Modal app – handles training, evaluation, and batch inference.
Run with:
    modal run modal_app.py::train
    modal run modal_app.py::evaluate
    modal run modal_app.py::transcribe_batch
"""

import modal
from pathlib import Path

```

```
# — Modal App Definition —————
app = modal.App("whisper-urdu-poc")

# Persistent volume — survives between runs
volume = modal.Volume.from_name("whisper-training-vol", create_if_missing=True)
VOLUME_PATH = "/data"

# Container image with all ML dependencies
image = (
    modal.Image.debian_slim(python_version="3.11")
    .apt_install("ffmpeg", "git")
    .pip_install([
        "transformers>=4.40.0",
        "datasets>=2.18.0",
        "evaluate>=0.4.0",
        "jiwer>=3.0.3",
        "torch>=2.2.0",
        "torchaudio>=2.2.0",
        "accelerate>=0.28.0",
        "soundfile>=0.12.1",
        "librosa>=0.10.1",
        "tensorboard>=2.16.0",
        "pyyaml>=6.0",
    ])
)

# — Training Function —————
@app.function(
    image=image,
    gpu="A10G",          # 24GB VRAM — perfect for Whisper medium
    timeout=60 * 60 * 8, # 8 hour max (10hrs audio ≈ 3-4hrs training)
    volumes={VOLUME_PATH: volume},
    memory=32768,        # 32GB RAM
)
def train():
    """Fine-tune Whisper medium on the prepared dataset."""
    import yaml
    import torch
    import numpy as np
    from dataclasses import dataclass
    from typing import Any, Dict, List, Union

    from datasets import load_from_disk, Audio
    from transformers import (
        WhisperProcessor,
        WhisperForConditionalGeneration,
```

```

        Seq2SeqTrainingArguments,
        Seq2SeqTrainer,
    )
import evaluate

# Load config
with open(f"{VOLUME_PATH}/config/training_config.yaml") as f:
    cfg = yaml.safe_load(f)

model_name = cfg["model"]["name"]
language = cfg["model"]["language"]
task = cfg["model"]["task"]
t_cfg = cfg["training"]

print(f"🚀 Loading {model_name}...")
processor = WhisperProcessor.from_pretrained(
    model_name, language=language, task=task
)
model = WhisperForConditionalGeneration.from_pretrained(model_name)
model.config.forced_decoder_ids = None
model.config.suppress_tokens = []

# Enable gradient checkpointing to reduce VRAM usage
model.config.use_cache = False

# — Data Collator —————
@dataclass
class DataCollatorSpeechSeq2SeqWithPadding:
    processor: Any
    decoder_start_token_id: int

    def __call__(self, features: List[Dict[str, Union[List[int], torch.Tensor]]]) \
        input_features = [
            {"input_features": f["input_features"]} for f in features
        ]
        batch = self.processor.feature_extractor.pad(
            input_features, return_tensors="pt"
        )
        label_features = [{"input_ids": f["labels"]} for f in features]
        labels_batch = self.processor.tokenizer.pad(
            label_features, return_tensors="pt"
        )
        labels = labels_batch["input_ids"].masked_fill(
            labels_batch.attention_mask.ne(1), -100
        )
        if (labels[:, 0] == self.decoder_start_token_id).all().cpu().item():

```

```

        labels = labels[:, 1:]
        batch["labels"] = labels
        return batch

# — Load Dataset —————
print("📁 Loading dataset from volume...")
dataset = load_from_disk(cfg["data"]["dataset_path"])
dataset = dataset.cast_column("audio", Audio(sampling_rate=16000))

def prepare_dataset(batch):
    audio = batch["audio"]
    batch["input_features"] = processor.feature_extractor(
        audio["array"],
        sampling_rate=audio["sampling_rate"]
    ).input_features[0]
    batch["labels"] = processor.tokenizer(batch["sentence"]).input_ids
    return batch

print("⚙️ Preprocessing audio features...")
dataset = dataset.map(
    prepare_dataset,
    remove_columns=dataset["train"].column_names,
    num_proc=4
)

# — Metrics —————
wer_metric = evaluate.load("wer")

def compute_metrics(pred):
    pred_ids = pred.predictions
    label_ids = pred.label_ids
    label_ids[label_ids == -100] = processor.tokenizer.pad_token_id
    pred_str = processor.batch_decode(pred_ids, skip_special_tokens=True)
    label_str = processor.batch_decode(label_ids, skip_special_tokens=True)
    wer = 100 * wer_metric.compute(predictions=pred_str, references=label_str)
    return {"wer": wer}

# — Training Arguments —————
training_args = Seq2SeqTrainingArguments(
    output_dir=t_cfg["output_dir"],
    per_device_train_batch_size=t_cfg["per_device_train_batch_size"],
    per_device_eval_batch_size=t_cfg["per_device_eval_batch_size"],
    gradient_accumulation_steps=t_cfg["gradient_accumulation_steps"],
    learning_rate=t_cfg["learning_rate"],
    warmup_steps=t_cfg["warmup_steps"],
    max_steps=t_cfg["max_steps"],

```

```

gradient_checkpointing=t_cfg["gradient_checkpointing"],
fp16=t_cfg["fp16"],
evaluation_strategy=t_cfg["evaluation_strategy"],
eval_steps=t_cfg["eval_steps"],
save_steps=t_cfg["save_steps"],
save_total_limit=t_cfg["save_total_limit"],
load_best_model_at_end=t_cfg["load_best_model_at_end"],
metric_for_best_model=t_cfg["metric_for_best_model"],
greater_is_better=t_cfg["greater_is_better"],
predict_with_generate=t_cfg["predict_with_generate"],
generation_max_length=t_cfg["generation_max_length"],
logging_steps=t_cfg["logging_steps"],
report_to=t_cfg["report_to"],
logging_dir=t_cfg["logging_dir"],
)

data_collator = DataCollatorSpeechSeq2SeqWithPadding(
    processor=processor,
    decoder_start_token_id=model.config.decoder_start_token_id,
)

trainer = Seq2SeqTrainer(
    args=training_args,
    model=model,
    train_dataset=dataset["train"],
    eval_dataset=dataset["eval"],
    data_collator=data_collator,
    compute_metrics=compute_metrics,
    tokenizer=processor.feature_extractor,
)

print("🔥 Starting training...")
trainer.train()

print("💾 Saving model + processor to volume...")
model_save_path = f"{VOLUME_PATH}/model/whisper-medium-urdu-final"
trainer.save_model(model_save_path)
processor.save_pretrained(model_save_path)
volume.commit()

print(f"✅ Training complete. Model saved to {model_save_path}")

# — Batch Transcription —
@app.function(
    image=image,

```

```

    gpu="T4",                # Cheaper GPU – inference only
    timeout=60 * 60 * 6,
    volumes={VOLUME_PATH: volume},
)
def transcribe_batch(audio_paths: list[str]) -> list[dict]:
    """Transcribe a batch of audio files using the fine-tuned model."""
    import torch
    import librosa
    from transformers import WhisperProcessor, WhisperForConditionalGeneration

    model_path = f"{VOLUME_PATH}/model/whisper-medium-urdu-final"
    processor = WhisperProcessor.from_pretrained(model_path)
    model = WhisperForConditionalGeneration.from_pretrained(model_path)
    model = model.to("cuda")
    model.eval()

    forced_decoder_ids = processor.get_decoder_prompt_ids(
        language="ur", task="transcribe"
    )
    results = []

    for audio_path in audio_paths:
        audio, sr = librosa.load(audio_path, sr=16000)
        inputs = processor(audio, sampling_rate=16000, return_tensors="pt")
        input_features = inputs.input_features.to("cuda")

        with torch.no_grad():
            predicted_ids = model.generate(
                input_features,
                forced_decoder_ids=forced_decoder_ids,
                max_new_tokens=225
            )

        transcript = processor.batch_decode(
            predicted_ids, skip_special_tokens=True
        )[0]
        results.append({"path": audio_path, "transcript": transcript})

    return results

```

5. `src/dataset_builder.py` — HuggingFace Dataset Builder

python

```
"""
```

```
Run locally to convert your manifest.json into a HuggingFace dataset,  
then upload to Modal Volume.
```

```
"""
```

```
import json  
from pathlib import Path  
from datasets import Dataset, DatasetDict, Audio  
  
def build_dataset(manifest_path: str, output_path: str,  
                  eval_split: float = 0.10):  
    with open(manifest_path, encoding="utf-8") as f:  
        samples = json.load(f)  
  
    # Shuffle consistently  
    import random  
    random.seed(42)  
    random.shuffle(samples)  
  
    split_idx = int(len(samples) * (1 - eval_split))  
    train_samples = samples[:split_idx]  
    eval_samples = samples[split_idx:]  
  
    def to_hf_format(sample_list):  
        return {  
            "audio": [s["audio_path"] for s in sample_list],  
            "sentence": [s["transcript"] for s in sample_list],  
            "language": [s.get("language", "ur") for s in sample_list],  
        }  
  
    dataset = DatasetDict({  
        "train": Dataset.from_dict(to_hf_format(train_samples)),  
        "eval": Dataset.from_dict(to_hf_format(eval_samples)),  
    })  
  
    # Cast audio column to load actual arrays  
    dataset = dataset.cast_column("audio", Audio(sampling_rate=16000))  
    dataset.save_to_disk(output_path)  
  
    print(f"✅ Dataset saved: {len(train_samples)} train / {len(eval_samples)} eval")  
    return dataset  
  
if __name__ == "__main__":  
    build_dataset(
```

```
manifest_path="./data/processed/manifest.json",  
output_path="./data/processed/dataset"  
)
```

6. End-to-End Workflow

```
bash
```

```
# — Step 1: Local Setup —————
```

```
pip install modal
```

```
modal token new          # Authenticate with Modal
```

```
claude                   # Launch Claude Code in project root
```

```
# — Step 2: Prepare Data Locally (via Claude Code) —————
```

```
python src/data_prep.py \  
  --input_dir ./raw_videos \  
  --transcript_dir ./transcripts \  
  --output_dir ./data/processed
```

```
python src/dataset_builder.py
```

```
# — Step 3: Upload to Modal Volume —————
```

```
modal volume put whisper-training-vol ./data/processed /data/processed
```

```
modal volume put whisper-training-vol ./config /data/config
```

```
# — Step 4: Train —————
```

```
modal run modal_app.py::train
```

```
# Monitor logs live – training runs in Modal cloud
```

```
# Checkpoints auto-save to Volume every 500 steps
```

```
# — Step 5: Download Best Model —————
```

```
modal volume get whisper-training-vol \  
  /data/model/whisper-medium-urdu-final ./model/final
```

```
# — Step 6: Batch Transcribe All 4000 Videos —————
```

```
modal run modal_app.py::transcribe_batch
```

Key Decisions for Your POC

Decision	Recommendation	Why
Language token	`<	ur
Transcript script	Roman Urdu + Arabic in Arabic script	Practical for code-switching; be 100% consistent in training data
GPU	A10G on Modal	24GB VRAM, ~\$1.10/hr — trains Whisper medium comfortably
Data for POC	Start with 2-3 hours	Validate WER improvement before committing to full labeling
Chunking	≤28s per chunk	Leave 2s buffer under Whisper's 30s hard limit
Batch size	16 + gradient_accum 2	Effective batch of 32 — stable training on A10G

The `CLAUDE.md` file ensures Claude Code understands the entire stack and can help you debug training runs, fix data pipeline issues, and iterate on the config without you having to re-explain context each session.

Would you like help with the transcript format strategy for the code-switching content, or the `scripts/upload_data.py` helper?

which model is better, whisper medium or fast whisper ?

Clarified that FastWhisper optimizes Whisper inference, not a separate model >

This is an important clarification — **they're not competing models**. You're comparing two different things:

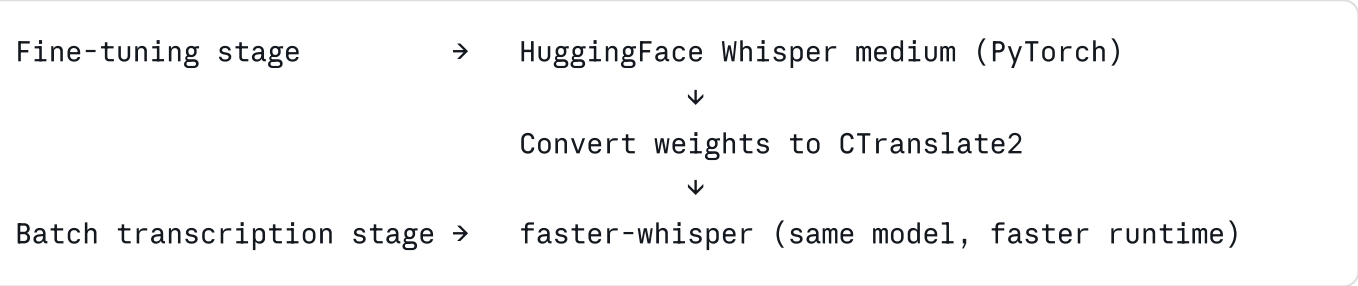
What They Actually Are

	Whisper Medium	faster-whisper
What it is	The actual ML model (300M params) by OpenAI	An optimized inference engine for the same Whisper model
Underlying weights	OpenAI's Whisper weights	Same OpenAI Whisper weights

	Whisper Medium	faster-whisper
Framework	PyTorch / HuggingFace	CTranslate2 (C++ runtime)
Accuracy	Baseline	Identical — same model
Inference speed	1x	2-4x faster
VRAM usage	~3GB (medium)	~1.5-2GB (with INT8)
Quantization	No	INT8, INT4 supported

The Key Insight for Your Project

For your 4000+ video pipeline, the right approach is to **use both**, at different stages:



Converting Your Fine-tuned Model for faster-whisper

After training, one extra step:

```
bash

pip install ctranslate2

# Convert HuggingFace checkpoint → CTranslate2 format
ct2-transformers-converter \
  --model ./model/final \           # your fine-tuned model path
  --output_dir ./model/ct2 \
  --quantization int8 \             # cuts VRAM in half, negligible accuracy loss
  --force
```

Then use it:

```
python

from faster_whisper import WhisperModel

# Load your fine-tuned model via faster-whisper
model = WhisperModel(
    "./model/ct2",
```

```
device="cuda",
compute_type="int8_float16"  # best speed/accuracy balance on GPU
)

segments, info = model.transcribe(
    "audio.wav",
    language="ur",
    task="transcribe",
    beam_size=5,
    vad_filter=True,          # built-in silence filtering – great for video
    vad_parameters={"min_silence_duration_ms": 500}
)

transcript = " ".join(seg.text for seg in segments)
```

Why faster-whisper Matters for Your 4000+ Video Batch

On an A10G GPU (Modal), rough numbers for **Whisper medium**:

Engine	Time per hour of audio	4000 videos (~30 min avg)
HuggingFace Whisper	~6–8 min	~200–270 hrs
faster-whisper (INT8)	~2–3 min	~67–100 hrs
faster-whisper (batched)	~1–1.5 min	~33–50 hrs

The batched inference in faster-whisper is particularly powerful:

```
python

# Batch multiple audio files in one GPU call
model = WhisperModel("./model/ct2", device="cuda", compute_type="int8_float16")

# Process multiple files concurrently
from faster_whisper import BatchedInferencePipeline

batched_model = BatchedInferencePipeline(model=model)
segments, info = batched_model.transcribe("audio.wav", batch_size=16)
```

Recommendation for Your POC

Fine-tune with HuggingFace (as set up in your `modal_app.py`), then **add a conversion + faster-whisper inference step** to your Modal pipeline. You get the accuracy gains from fine-tuning *and* the speed gains from CTranslate2 — best of both worlds.

Want me to update the `modal_app.py` to include the conversion step and a faster-whisper batch transcription function?

Question, Let's say we train a whisper medium model in Modal Platform, will we be able to reuse it outside the platform ?

Clarified Modal's portability and confirmed model weights remain universally accessible >

Yes, absolutely — and this is one of Modal's strengths. It's purely a **compute platform**, not a model-locking ecosystem.

What Modal Actually Stores

When training completes, your fine-tuned model is saved to a **Modal Volume** as standard files:

```
/data/model/whisper-medium-urdu-final/  
├─ config.json  
├─ model.safetensors      ← just PyTorch weights  
├─ tokenizer.json  
├─ tokenizer_config.json  
├─ vocab.json  
├─ merges.txt  
└─ processor_config.json
```

These are plain HuggingFace format files — no Modal-specific format, no lock-in.

Getting Your Model Out

```
bash  
  
# Download from Modal Volume to your local machine anytime  
modal volume get whisper-training-vol \  
  /data/model/whisper-medium-urdu-final \  
  ./my-local-model
```

That's it. The model is now yours locally.

Where You Can Use It After Downloading

Destination	How
Local machine	<code>WhisperModel("./my-local-model")</code> via HuggingFace or faster-whisper
Your own GPU server	Same as local
AWS / GCP / Azure	Upload weights, run inference there
HuggingFace Hub	<code>model.push_to_hub("your-org/whisper-medium-urdu")</code> — private or public
On-premise	Copy files, run anywhere Python runs
Any other cloud	Replicate, RunPod, Lambda Labs, etc.

Pushing to HuggingFace Hub (Recommended)

HuggingFace Hub acts as your permanent model registry — much safer than keeping it only on a Modal Volume:

```
python

from transformers import WhisperForConditionalGeneration, WhisperProcessor

model = WhisperForConditionalGeneration.from_pretrained("./my-local-model")
processor = WhisperProcessor.from_pretrained("./my-local-model")

# Push to HuggingFace (set private=True to keep it internal)
model.push_to_hub("your-org/whisper-medium-urdu", private=True)
processor.push_to_hub("your-org/whisper-medium-urdu", private=True)
```

Then load it anywhere in the world with just:

```
python

model = WhisperForConditionalGeneration.from_pretrained("your-org/whisper-medium-urdu")
```

One Thing to Be Aware Of

Modal Volumes are **not permanent by default** — if you delete the volume or your account lapses, the data is gone. So the recommended flow is:

Train on Modal → Save to Modal Volume



Download locally + Push to HuggingFace Hub (private)



Use anywhere, anytime — Modal is no longer needed

Think of Modal purely as the **GPU rental** for training. The model artifact it produces is entirely yours and fully portable.

